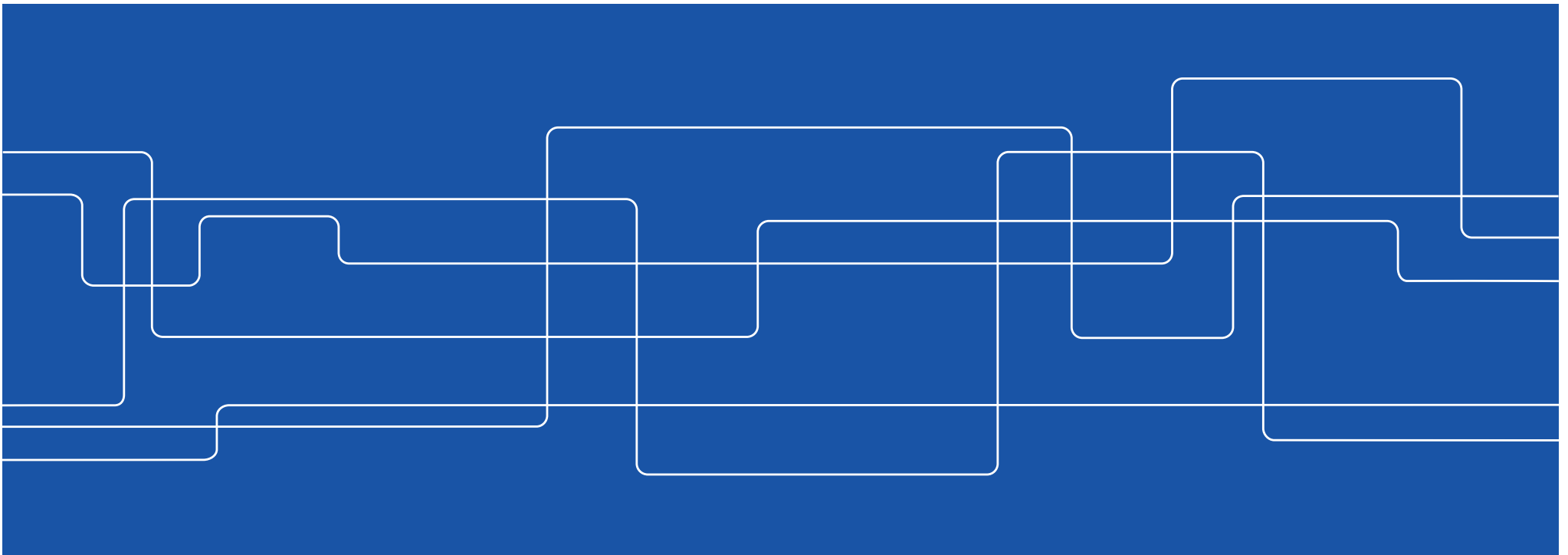


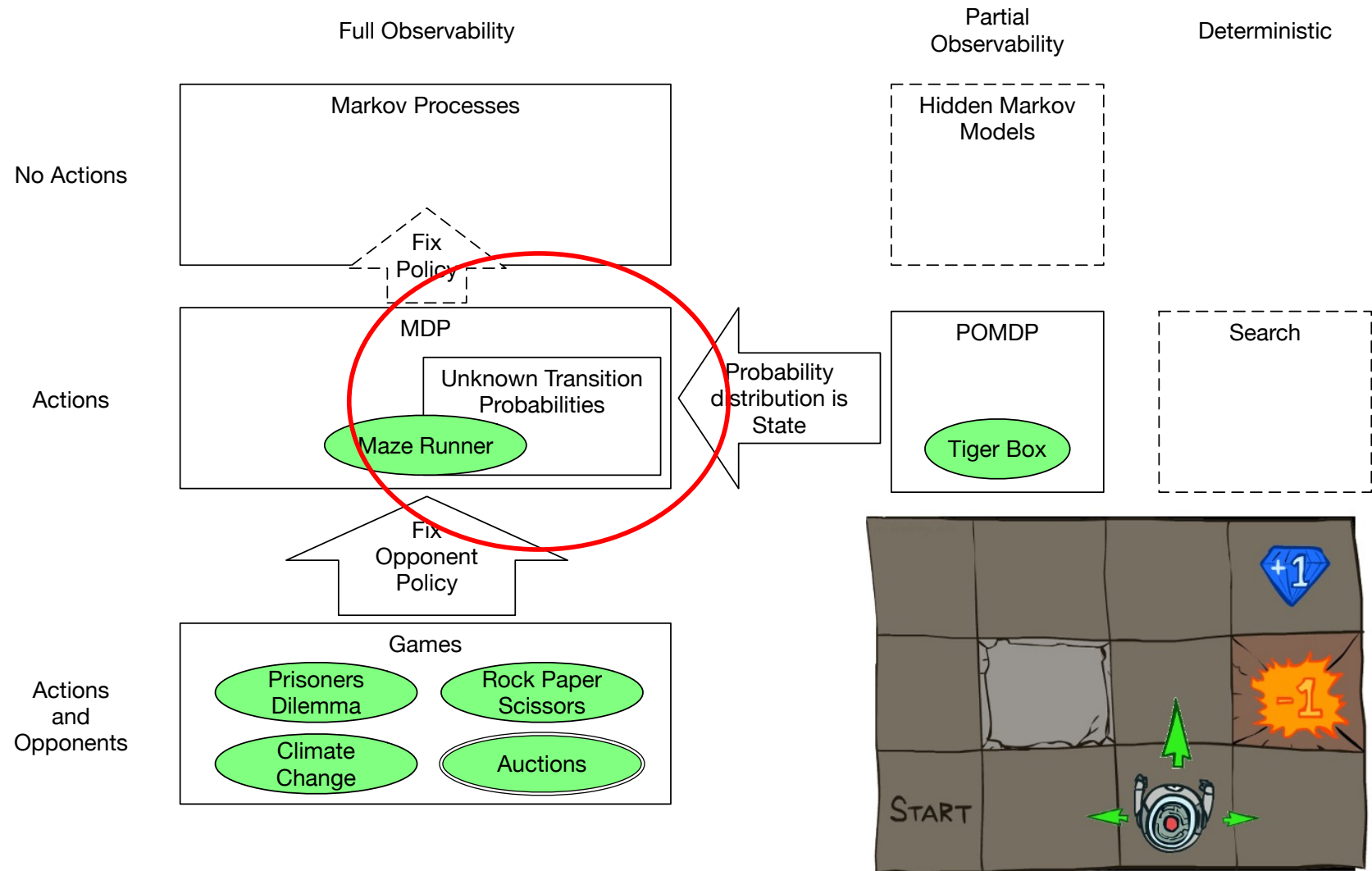


DD2380 Artificial Intelligence

Reinforcement Learning
Petter Ögren



Problem Classes





- $U(s)$: Utility of following policy
- $Q(s,a)$: Utility of *first doing a*, then following *policy*

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

$$a(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s')$$

$$U(s) = \max_a Q(s, a)$$

$$a(s) = \operatorname{argmax}_a Q(s, a)$$

What to do
without known
Transition
Probabilities or
Rewards
functions?

0.67	0.81	0.94	1.00
0.54		0.62	-1.00
0.41	0.32	0.45	0.19

OPTIMAL VALUES (VALUE ITERATION)

0.58	0.71	0.84	1.00
0.56	0.67	0.60	0.81
0.48	0.71	0.59	0.94
0.54		0.62	-1.00
0.44	0.44	0.53	-0.68
0.34		0.23	
0.41	0.24	0.45	-0.76
0.32	0.25	0.29	0.32
0.26	0.16	0.19	-0.02
0.30	0.24	0.31	0.11

OPTIMAL Q-VALUES (VALUE ITERATION)



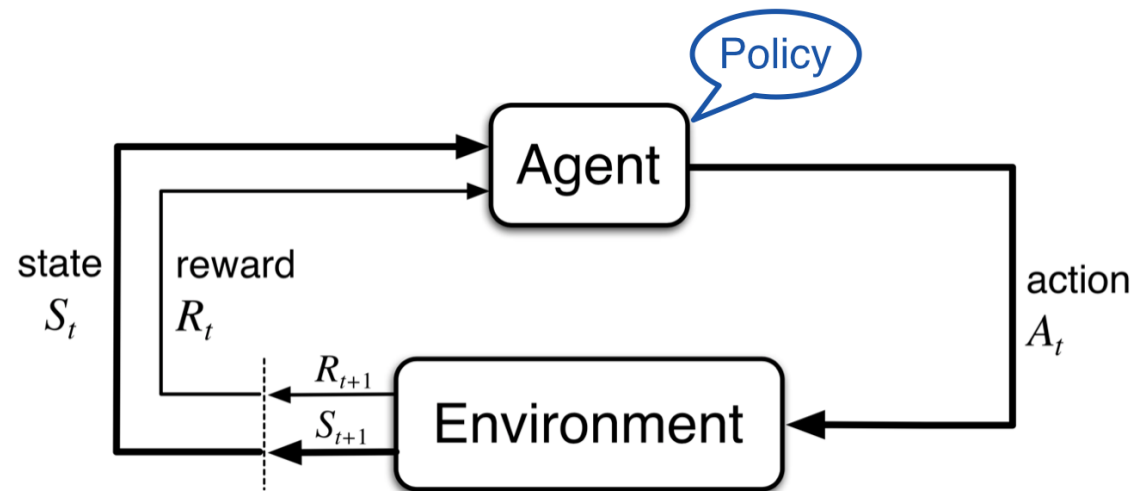
What is Reinforcement Learning?

Rewards provide a positive reinforcement to our actions





What is Reinforcement Learning?

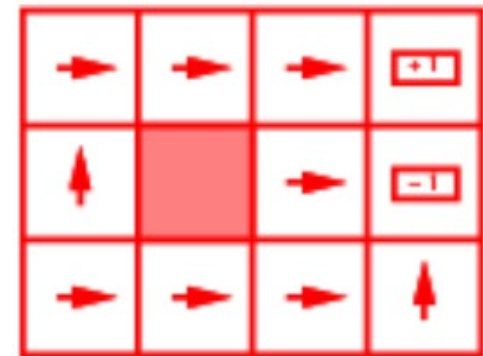


- Goal-directed **learning** from **interaction**
- Feedback in the form of **rewards**
- Agent must (learn to) act so as to **maximize expected rewards**
- Learning is based on observed samples of outcomes



Reinforcement Learning vs Supervised Learning

- No supervisor comparing to given “Correct answer”
- Only reward signals
- Sequential decisions
- Feedback is often delayed (to the end)





RL Applications

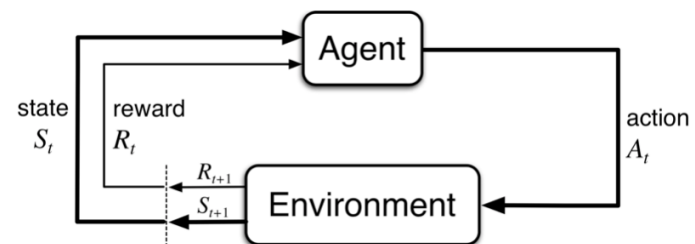
- Robot control (e.g. drones)
- Game playing
 - Playing Atari games, Tetris, etc.
- Medical treatment
 - Planning a sequence of treatments based on the effect of past treatments
- Chat bots (e.g. Siri, Alexa, ChatGPT, Gemini...)
 - Aligning to human preferences (RL from Human Feedback)



MDP Formulation

Mathematical model: Markov Decision Process (S, A, T, R, γ)

- State space S
- Action space A
- Environment Transition model T
- Reward function R
- Discount factor γ



$$T(s, a, s') : p(s_{t+1} = s' | s_t = s, a_t = a)$$

$R(s, a, s')$: immediate reward for transitioning from state s to state s' due to action a

Frequently reward depends only on the state, so we usually write $R(s)$

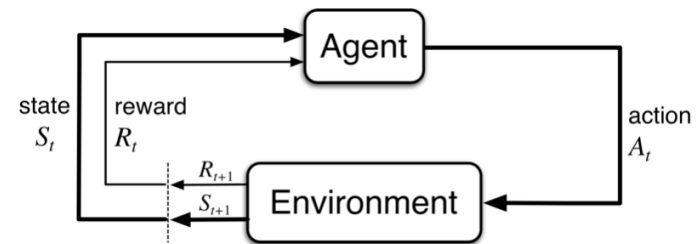
From lecture:
"Making Decisions
Under Uncertainty"



RL Formulation

- Still assume a Markov Decision Process (MDP):

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Environment Transition model T
- Reward function R



- Still looking for a policy $\pi(s)$
- In RL, new twist: **don't know T or R** , so:
 - We don't know which states are good or what the actions do
 - Agent must actually try actions and states to learn



RL Formulation

Goal: find an optimal policy that maximizes expected rewards

Recall the principle of maximum expected utility:

$$a(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

Where $U(s)$ satisfies Bellman optimality equation:

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

From lecture:
"Making Decisions
Under Uncertainty"



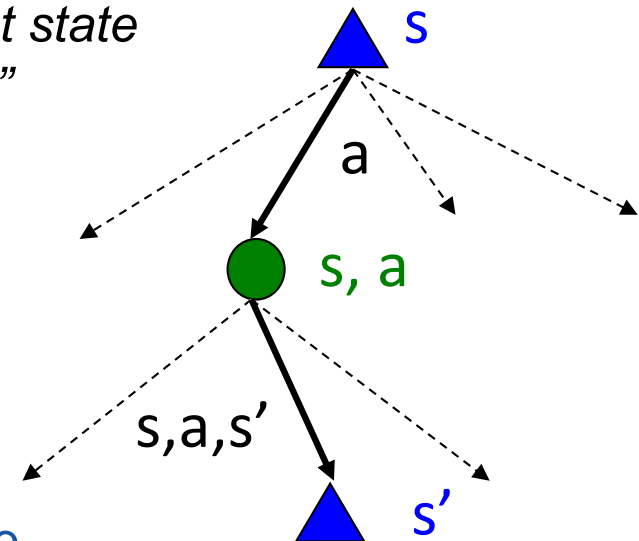
Bellman equation

“The utility of a state is the immediate reward for that state plus the expected discounted utility of the next state”

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

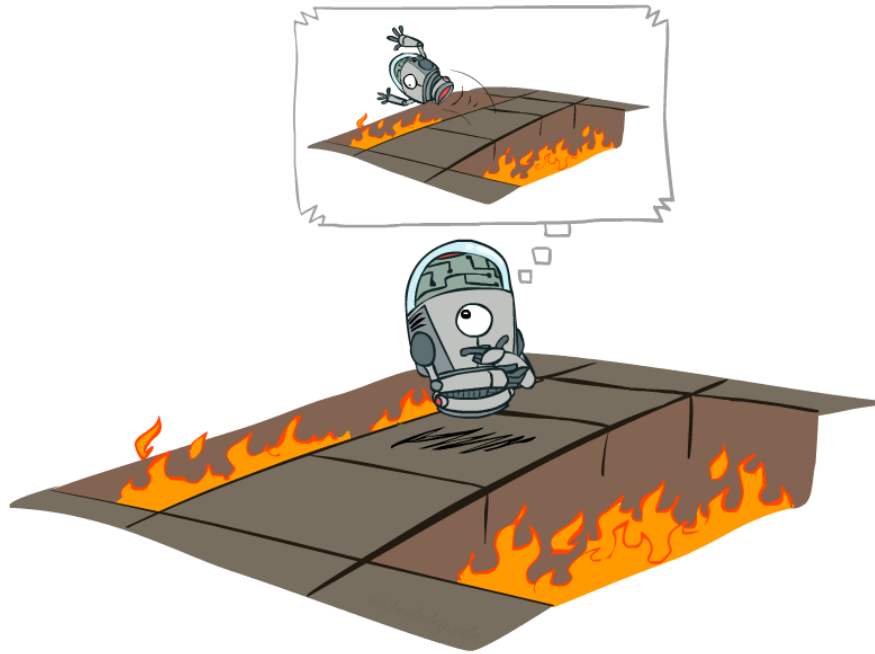
immediate reward

discounted expected utility of the next state, assuming optimal action



In RL, we don't know T!

Offline (MDPs) VS. Online (RL) Learning



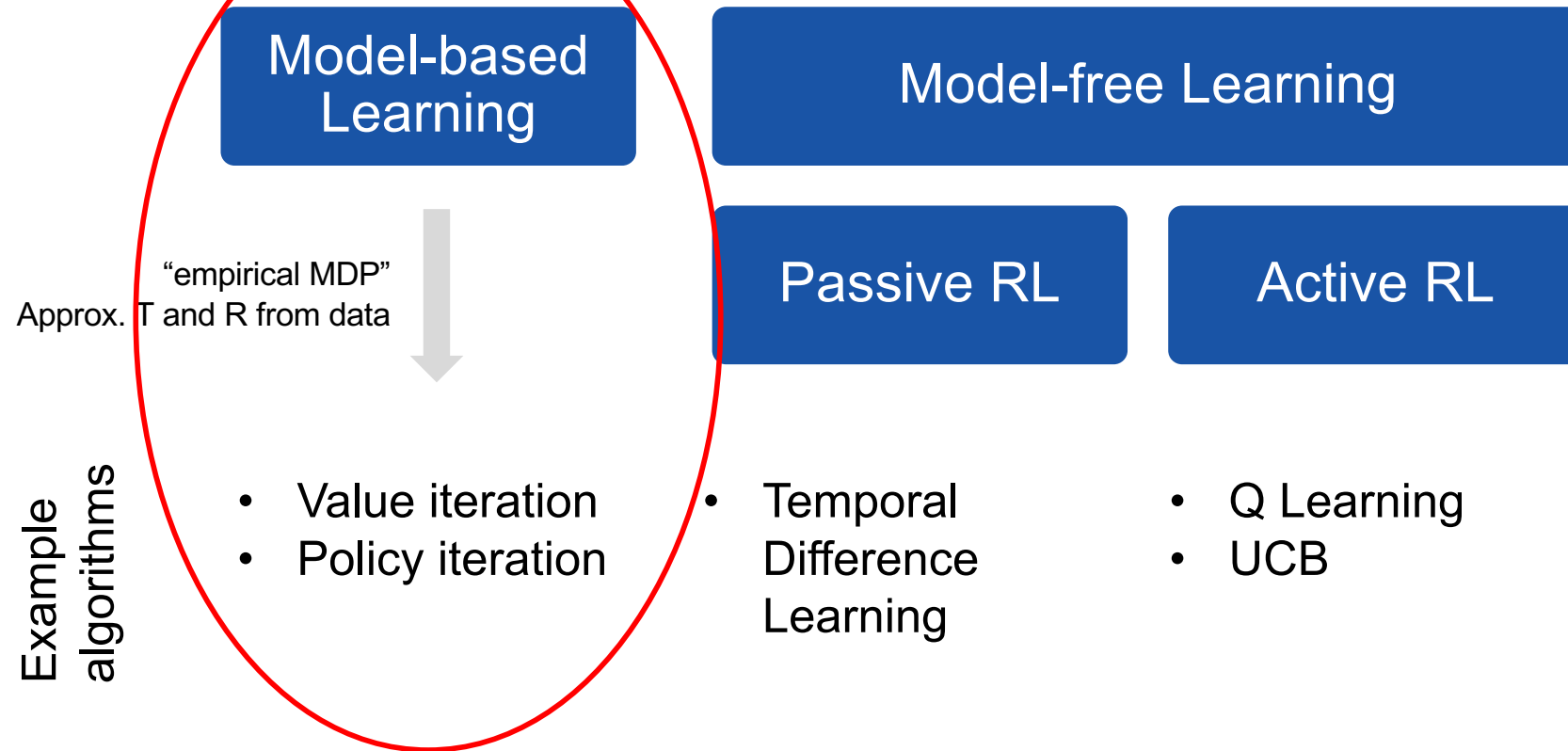
Offline Solution:
Known T and R



Online Learning:
need to estimate T and R



How to find optimal policies without knowing T and R ?





Model-Based Learning

Key intuition:

- Learn an approximate model based on experiences
- Solve for values as if the learned model were correct

Step 1: Learn empirical MDP model

- Count outcomes s' for each s, a
- Normalize to give an estimate of $\hat{T}(s, a, s')$
- Discover each $\hat{R}(s, a, s')$ when we experience (s, a, s')

$$\text{Probability} = \frac{\text{Count}(s, a, s')}{\text{Count}(s, a)}$$

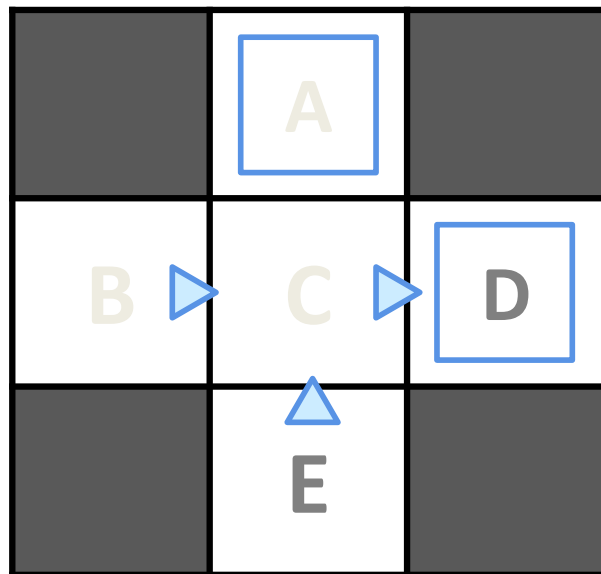
Step 2: Solve the learned MDP

- For example, use value iteration, as before



Model-Based Learning: Example

Input Policy π



Assume: $\gamma = 1$

Observed Episodes (Training)

Episode 1

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 2

B, east, C, -1
C, east, D, -1
D, exit, x, +10

Episode 3

E, north, C, -1
C, east, D, -1
D, exit, x, +10

Episode 4

E, north, C, -1
C, east, A, -1
A, exit, x, -10

Learned Model

$$\hat{T}(s, a, s')$$

$T(B, \text{east}, C) = 1.00$
 $T(C, \text{east}, D) = 0.75$
 $T(C, \text{east}, A) = 0.25$
...

$$\hat{R}(s, a, s')$$

$R(B, \text{east}, C) = -1$
 $R(C, \text{east}, D) = -1$
 $R(D, \text{exit}, x) = +10$
...

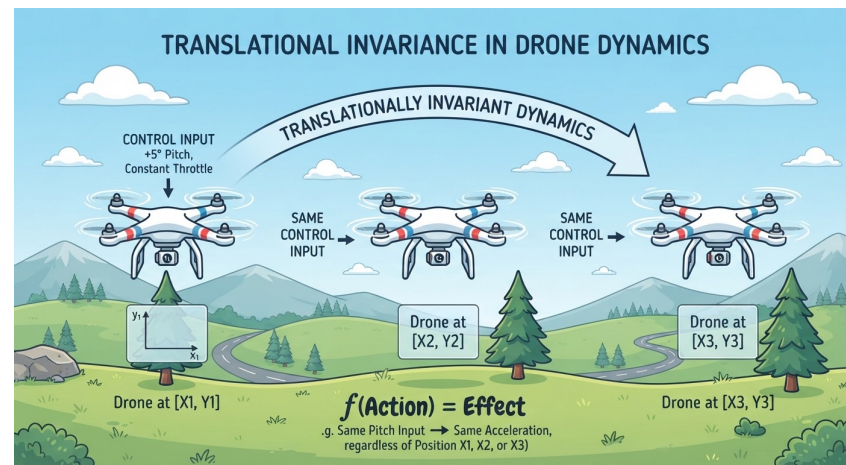
Model-Based Learning

Advantage:

- Make good use of data

Disadvantage:

- Requires building the actual MDP model
- Intractable if state space is too large



Can make clever choices.
Example: body centric coordinates

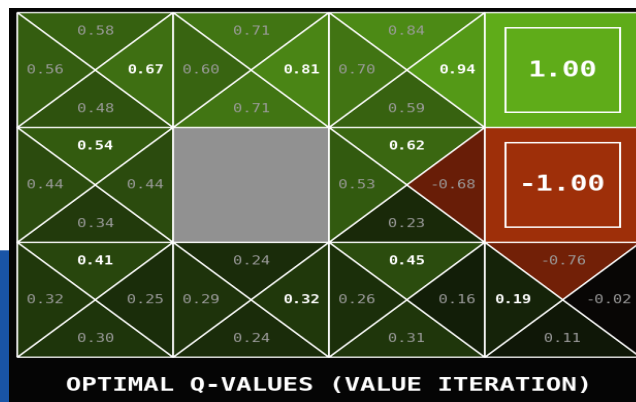


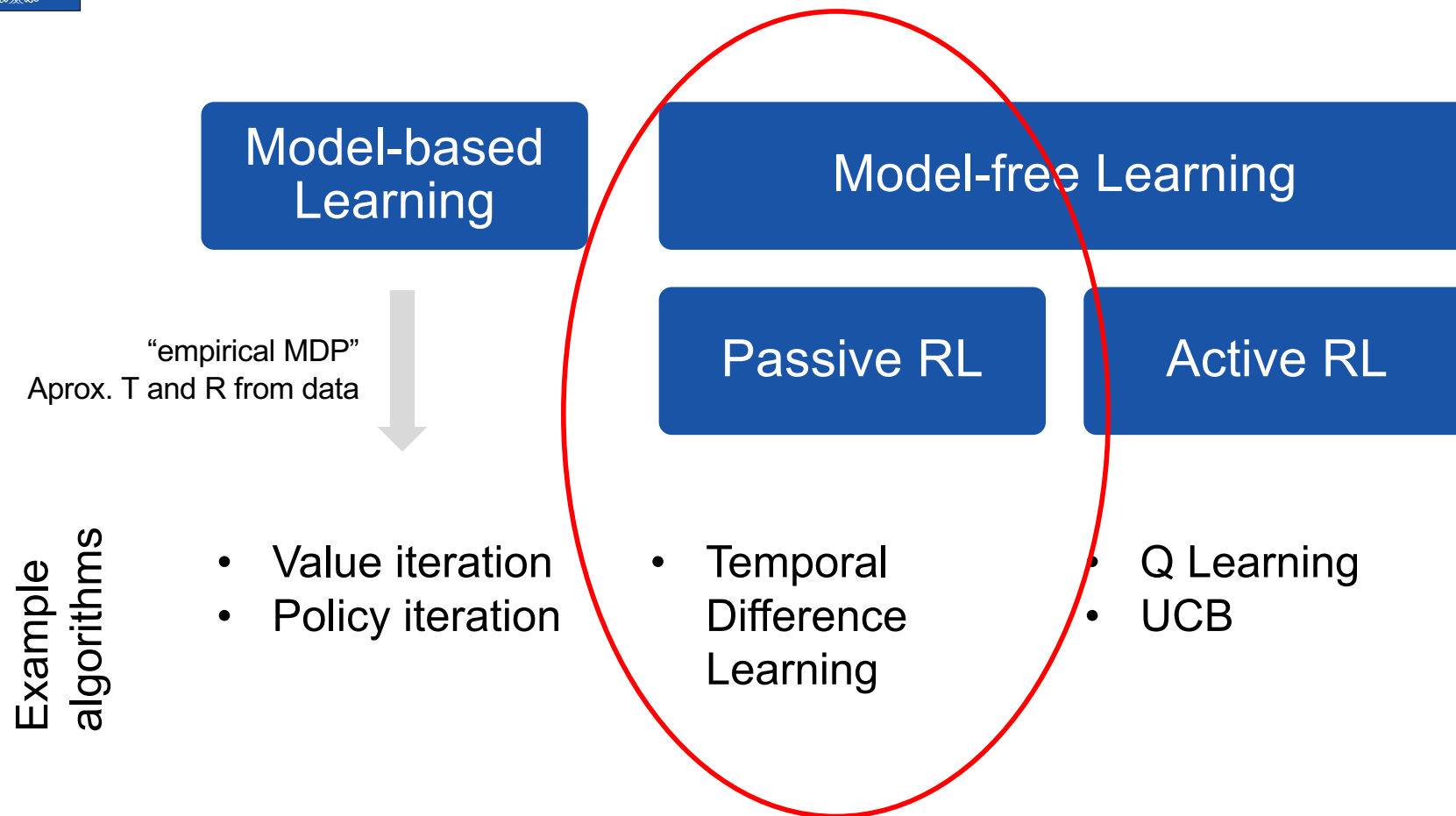
Next: Model-Free Learning

- Remember: we don't know the transition model $\mathbf{T}$ and reward distribution $\mathbf{R}$

Key intuition: *Learn while optimizing policy*

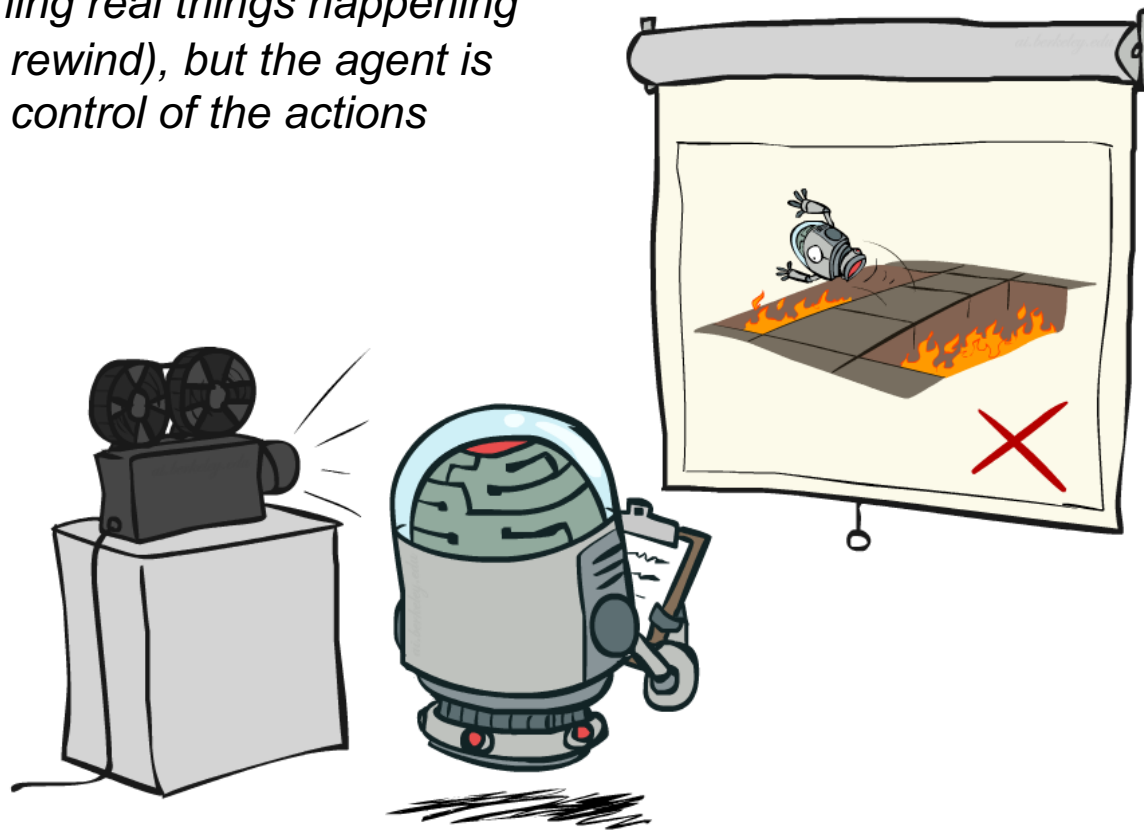
- Model is unknown but agent observes *samples*
- Learn optimal policy by sampling the environment (without an explicit model)
- $Q(s,a)$ will be useful...





Passive Reinforcement Learning

*Watching real things happening
(can't rewind), but the agent is
not in control of the actions*





Idea: Average approximates Expected value

We want this (but we don't have T)

$$U_{k+1}^{\pi}(s) \leftarrow R(s, \pi(s)) + \gamma \sum_{s'} T(s, \pi(s), s') U_k^{\pi}(s')$$

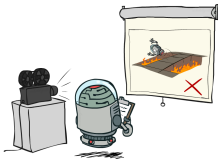
Using a single sample instead?

$$U_{k+1}^{\pi}(s) \leftarrow R(s, \pi(s)) + \gamma U_k^{\pi}(s')$$

Policy
Evaluation

Using an “Exponential Moving Average” of samples!

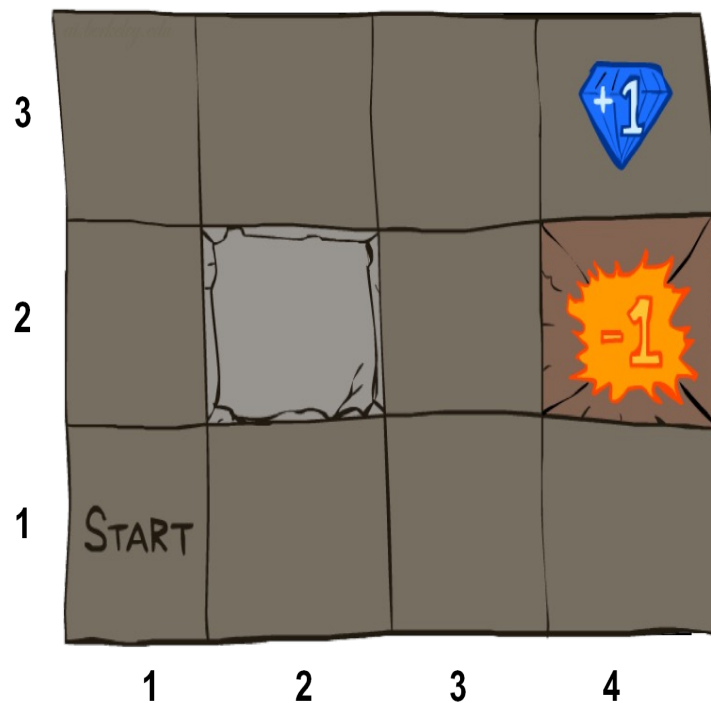
$$U_{k+1}^{\pi}(s) \leftarrow (1 - \alpha) U_k^{\pi}(s) + \alpha (R(s, \pi(s)) + \gamma U_k^{\pi}(s'))$$



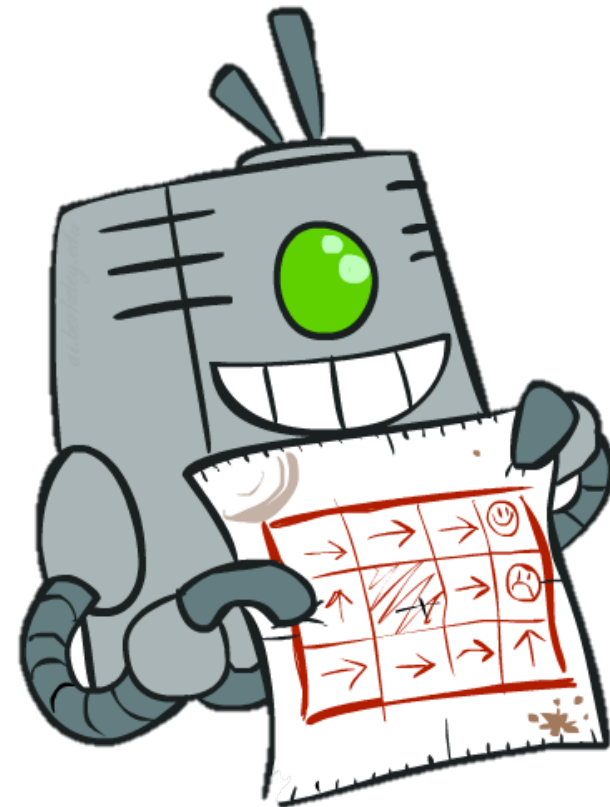
This is called: Passive Temporal Difference (TD) learning

TD Learning Example

You are in an environment (without knowing T and R)



You are given a policy π





TD Example

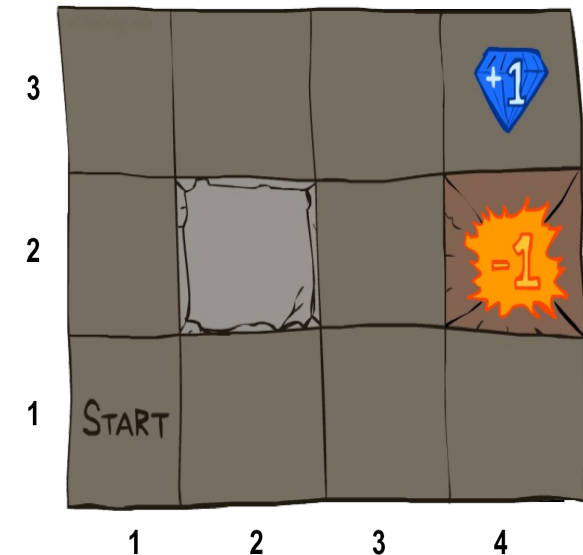
Initialize $V(s)$ to be 0

$$\alpha = 0.1, \gamma = 1$$

$$V(s) \leftarrow (1 - \alpha)V(s) + \alpha(r + \gamma V(s'))$$

State	$V(s)$
(1,1)	0
(1,2)	0
(1,3)	0
(4,1)	0
(2,1)	0
(2,3)	0
(3,1)	0
(3,2)	0
(3,3)	0

Environment



Policy





TD Example

$$\alpha = 0.1, \gamma = 1$$

$$V(s) \leftarrow (1 - \alpha)V(s) + \alpha(r + \gamma V(s'))$$

Start at (1,1)

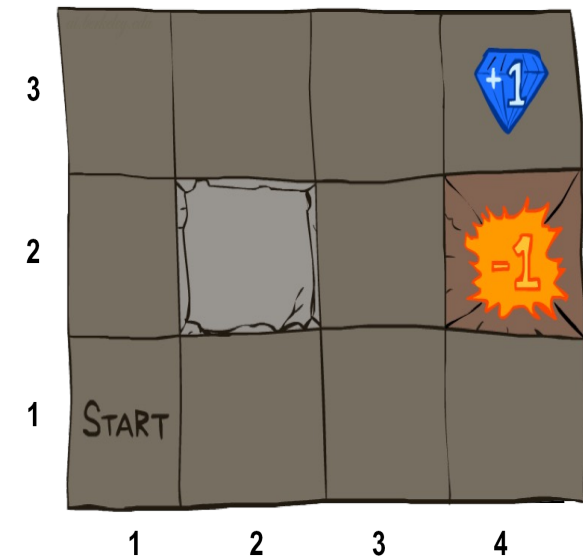
$s = (1,1)$ action=right

Reward = -0.01 ; End up at $s' = (2,1)$

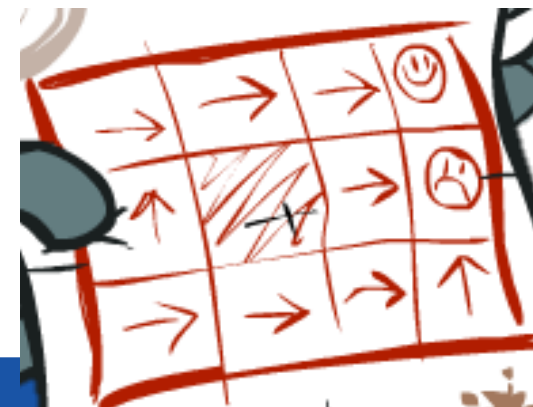
$$V((1,1)) = 0.9 * 0 + 0.1 * (-0.01 + 0) = -0.001$$

State	$V(s)$
(1,1)	0
(1,2)	0
(1,3)	0
(4,1)	0
(2,1)	0
(2,3)	0
(3,1)	0
(3,2)	0
(3,3)	0

Environment



Policy





TD Example

$$\alpha = 0.1, \gamma = 1$$

$$V(s) \leftarrow (1 - \alpha)V(s) + \alpha(r + \gamma V(s'))$$

Start at (1,1)

$s = (1,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

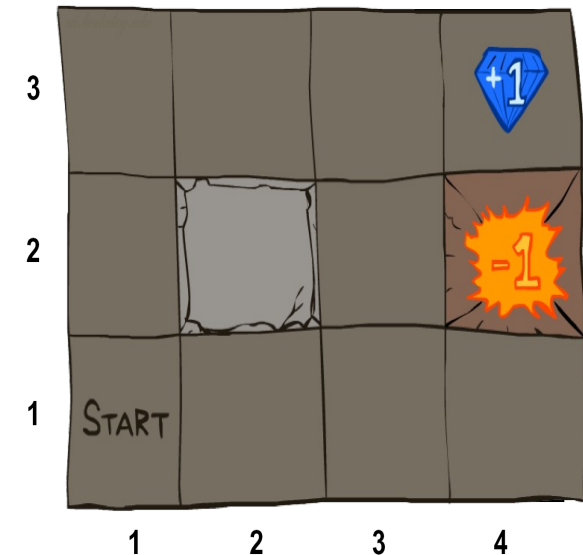
$s = (2,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

$$V((2,1)) = 0.9 * 0 + 0.1 * (-0.01 + 0) = -0.001$$

State	$V(s)$
(1,1)	-0.001
(1,2)	0
(1,3)	0
(4,1)	0
(2,1)	0
(2,3)	0
(3,1)	0
(3,2)	0
(3,3)	0

Environment



Policy





TD Example

$$\alpha = 0.1, \gamma = 1$$

$$V(s) \leftarrow (1 - \alpha)V(s) + \alpha(r + \gamma V(s'))$$

Start at (1,1)

$s = (1,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

$s = (2,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

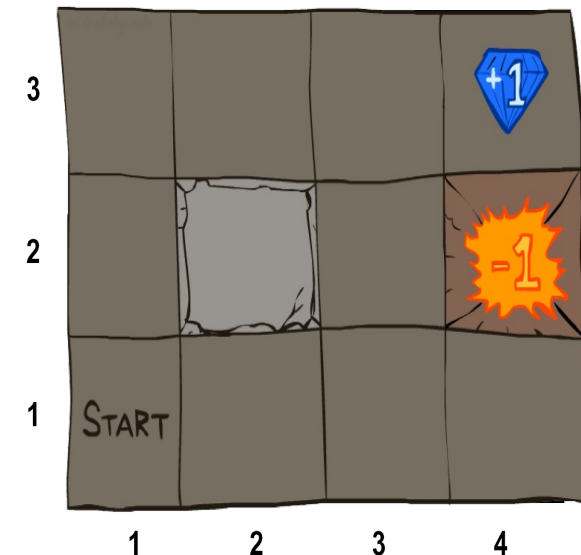
$s = (2,1)$ action=right

Reward= -0.01 ; End up at $s' = (3,1)$

$$V((2,1)) = 0.9 * -0.001 + 0.1 * (-0.01 + 0) = -0.0019$$

State	$V(s)$
(1,1)	-0.001
(1,2)	0
(1,3)	0
(4,1)	0
(2,1)	-0.001
(2,3)	0
(3,1)	0
(3,2)	0
(3,3)	0

Environment



Policy





TD Example

$$\alpha = 0.1, \gamma = 1$$

$$V(s) \leftarrow (1 - \alpha)V(s) + \alpha(r + \gamma V(s'))$$

Can we use the same trick to find optimal policy?

Start at (1,1)

$s = (1,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

$s = (2,1)$ action=right

Reward= -0.01 ; End up at $s' = (2,1)$

$s = (2,1)$ action=right

Reward= -0.01 ; End up at $s' = (3,1)$

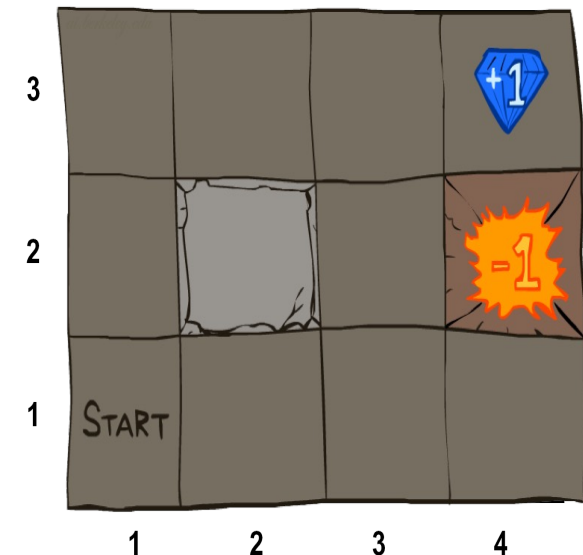
$s = (3,1)$ action=right

Reward= -0.01 ; End up at $s' = (4,1)$

$$V((3,1)) = 0.9 * 0 + 0.1 * (-0.01 + 0) = -0.001$$

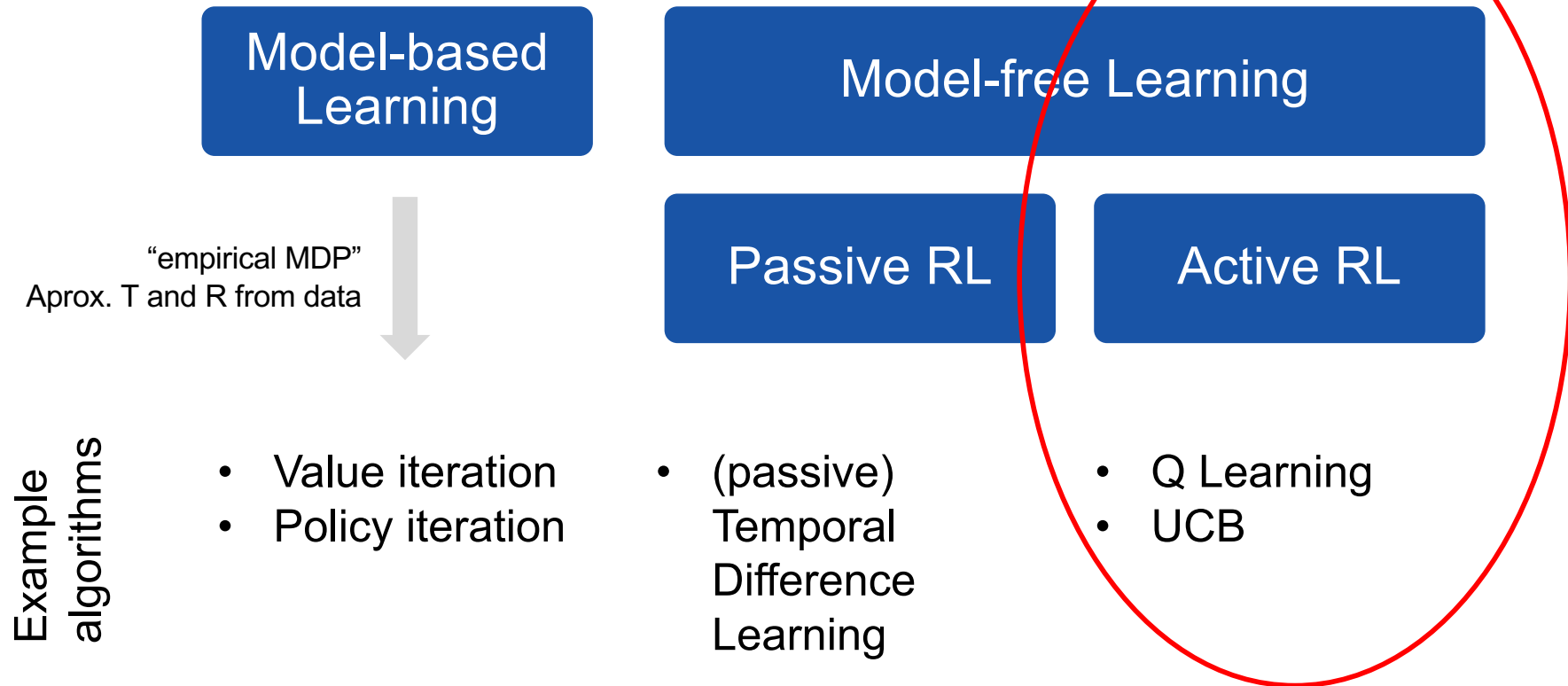
State	$V(s)$
(1,1)	-0.001
(1,2)	0
(1,3)	0
(4,1)	0
(2,1)	-0.0019
(2,3)	0
(3,1)	-0.001
(3,2)	0
(3,3)	0

Environment



Policy



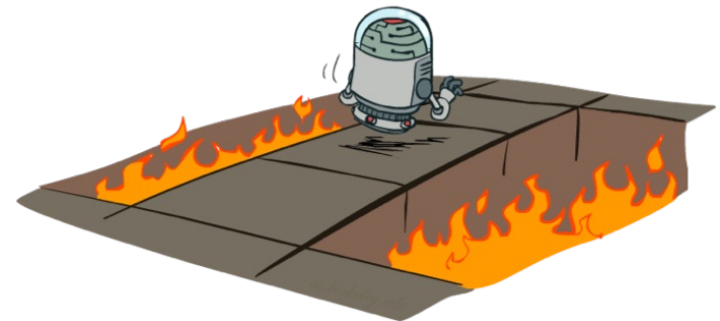




Active Reinforcement Learning

Full reinforcement learning: optimal policies

- You don't know the transitions $T(s,a,s')$
- You don't know the rewards $R(s,a,s')$
- You choose the actions now
- **Goal: learn the optimal policy / values**



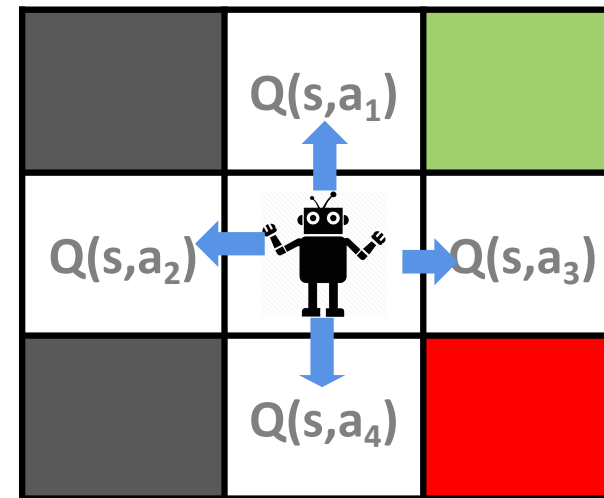
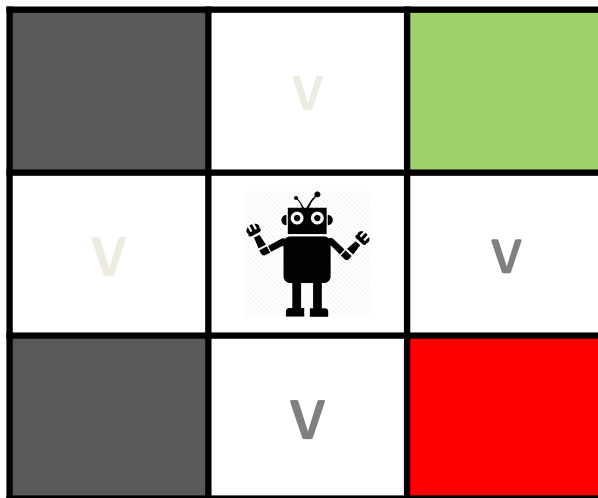
In this case:

- Learner makes choices!
- Unlike passive RL, in active RL the agent needs to learn the outcome probabilities for all actions, not just the model for the fixed policy
- Fundamental tradeoff: exploration vs. exploitation



Q-Learning Intuition

Optimal value functions (used e.g. in TD learning) tell us how “good” a **state** is, but does not explicitly capture how good the **actions** are



Instead of estimating the *value* of a state (V), estimate the *actions* $Q(s,a)$



- $U(s)$: Utility of following policy
- $Q(s,a)$: Utility of *first doing a*, then following *policy*

$$U(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$
$$a(s) = \operatorname{argmax}_a \left(R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s') \right)$$

$$U(s) = \max_a Q(s, a)$$
$$a(s) = \operatorname{argmax}_a Q(s, a)$$

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s')$$

Apply the moving average trick!

0.67	0.81	0.94	1.00
0.54		0.62	-1.00
0.41	0.32	0.45	0.19

OPTIMAL VALUES (VALUE ITERATION)

0.58	0.71	0.84	1.00
0.56	0.67	0.60	0.81
0.48	0.71	0.59	0.94
0.54		0.62	-1.00
0.44	0.44	0.53	-0.68
0.34		0.23	
0.41	0.24	0.45	-0.76
0.32	0.25	0.29	0.32
0.26	0.16	0.19	-0.02
0.30	0.24	0.31	0.11

OPTIMAL Q-VALUES (VALUE ITERATION)



Applying the “moving average trick”

Inspiration: Passive TD learning

$$U_{k+1}^{\pi}(s) \leftarrow (1 - \alpha)U_k^{\pi}(s) + \alpha(R(s, \pi(s)) + \gamma U_k^{\pi}(s'))$$

Def. of Q

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') U(s')$$

Getting rid of U

$$Q(s, a) = R(s, a) + \gamma \sum_{s'} T(s, a, s') \max_{a'} Q(s', a')$$

Moving average

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q_k(s', a'))$$

But how to choose action a?

0.67	0.81	0.94	1.00
0.54		0.62	-1.00
0.41	0.32	0.45	0.19

OPTIMAL VALUES (VALUE ITERATION)

0.58 0.56	0.71 0.67	0.84 0.60	1.00
0.48 0.54	0.71 0.44	0.59 0.62	-1.00
0.34 0.41	0.24 0.29	0.23 0.32	-0.76 0.19

OPTIMAL Q-VALUES (VALUE ITERATION)

- $U(s)$: Utility of following policy
- $Q(s,a)$: Utility of *first doing a*, then following *policy*



Q-Learning Algorithm

Initialize $Q(s, a) = 0 \forall s, a$

Loop:

Set $\pi_{\epsilon\text{-greedy}} = \begin{cases} \arg \max_a Q(s, a) & \text{with prob } (1 - \epsilon) \\ \text{a random action} & \text{with prob } \epsilon \end{cases}$

Take action $a = \pi_{\epsilon\text{-greedy}}(s)$ observe reward r & next state s'

Update $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

$$Q_{k+1}(s, a) \leftarrow (1 - \alpha)Q_k(s, a) + \alpha(R(s, a) + \gamma \max_{a'} Q(s', a'))$$

Re-arranged terms...

But how to chose
epsilon and alpha?



Choosing Alpha and Epsilon

Initialize $Q(s, a) = 0 \forall s, a$

Loop:

Set $\pi_{\epsilon\text{-greedy}} = \begin{cases} \arg \max_a Q(s, a) & \text{with prob } (1 - \epsilon) \\ \text{a random action} & \text{with prob } \epsilon \end{cases}$

Take action $a = \pi_{\epsilon\text{-greedy}}(s)$ observe reward r & next state s'

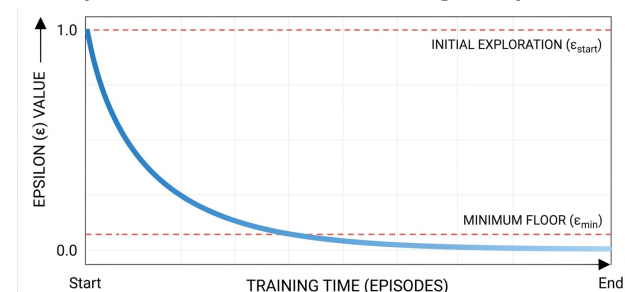
Update $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

Alpha

- 0.01-0.1 (Noisy & final stage)
- 0.1-0.5 (standard)
- 0.5-0.9 (almost deterministic)

Epsilon

- Decay towards non-zero (in case of changes)

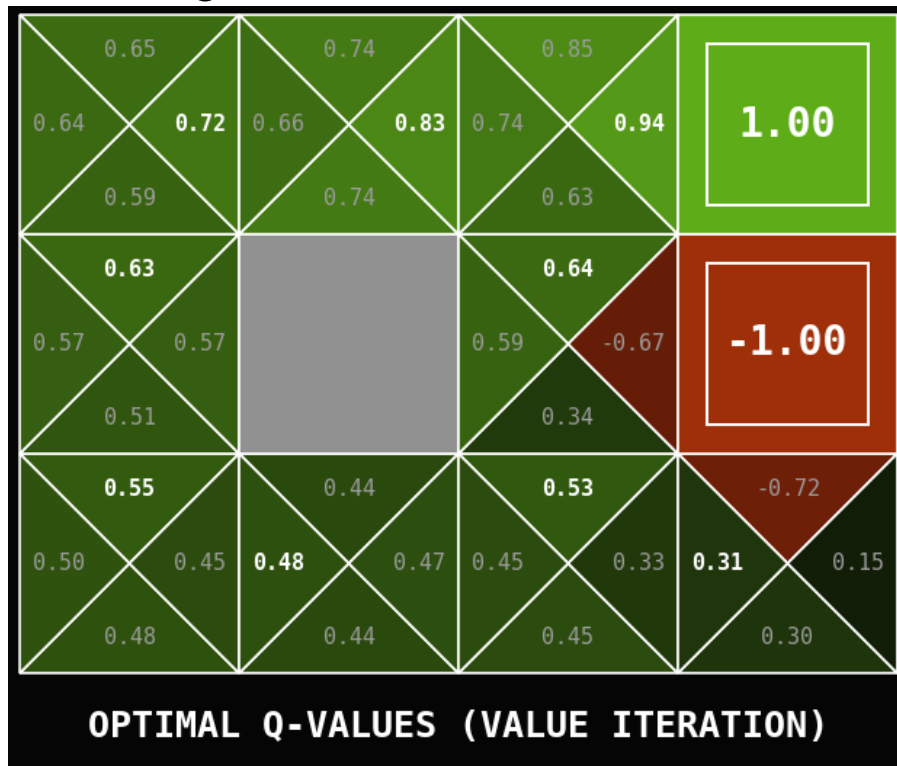




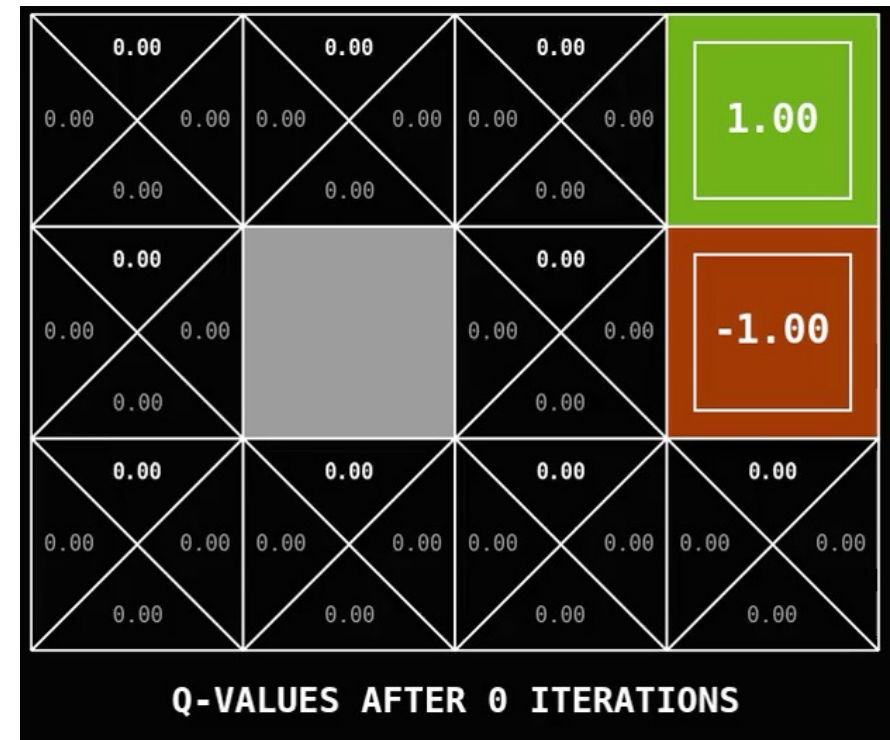
epsilon = 0.3
alpha = 0.05

Q-Learning on the Maze

Value Iteration (known T)
Converged after 29 iterations



Q-Learning (unknown T)
many iterations needed

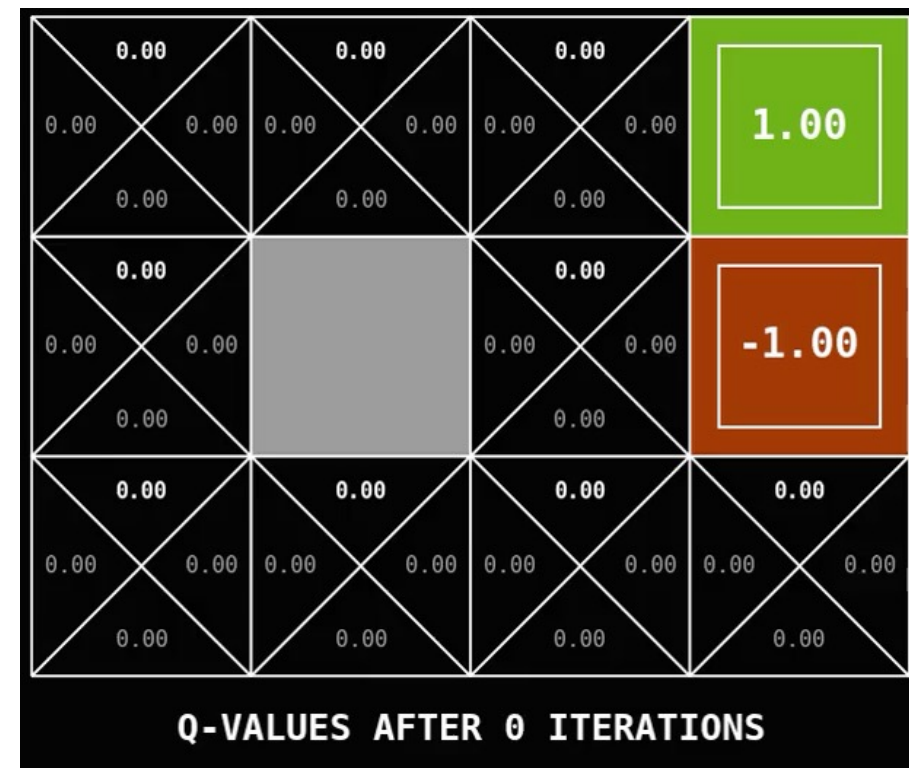
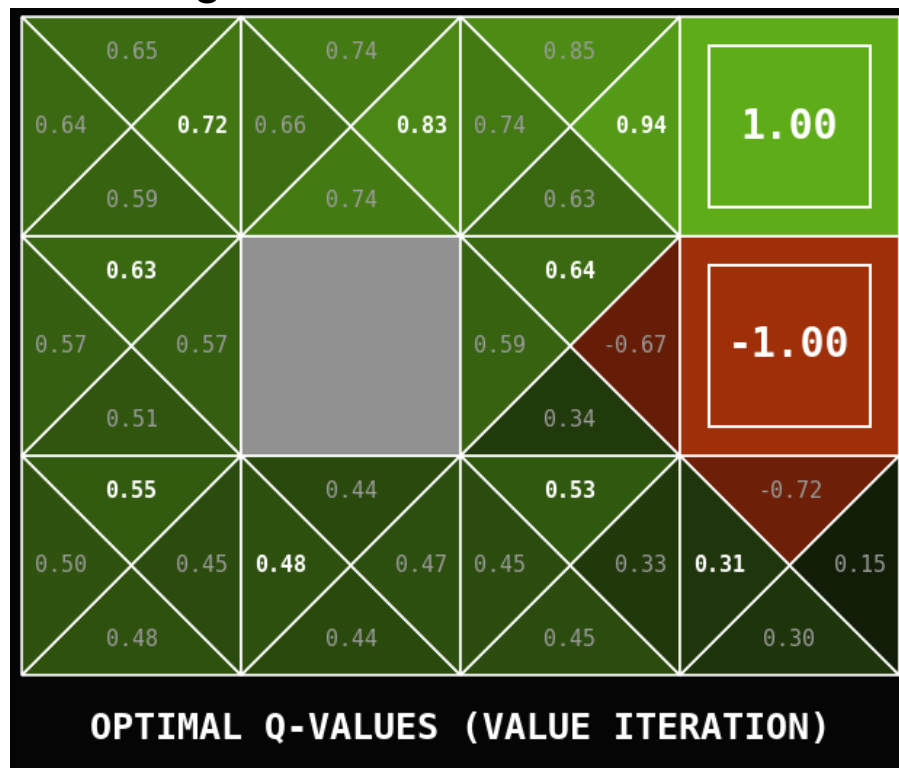




Q-Learning on the Maze (poor parameters)

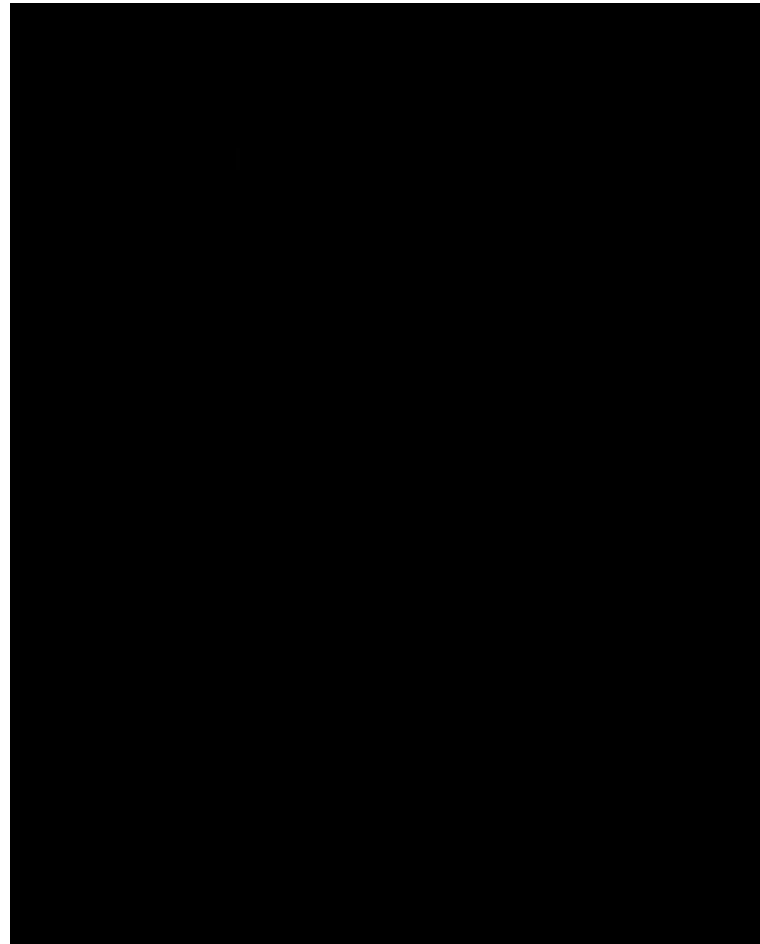
epsilon = 0.1
alpha = 0.5

Value Iteration (known T)
Converged after 29 iterations



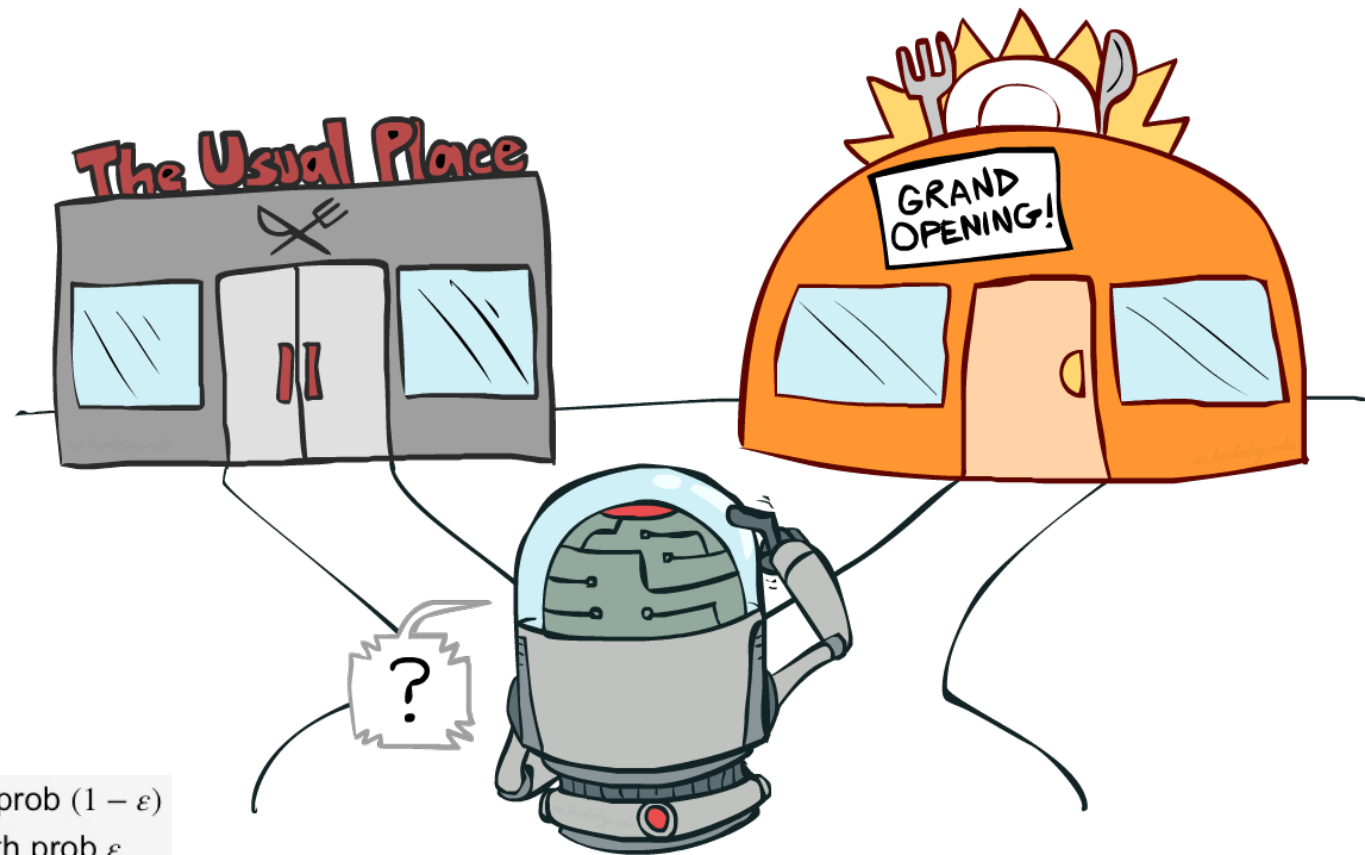


Q-Learning: Deep Mind ATARI example



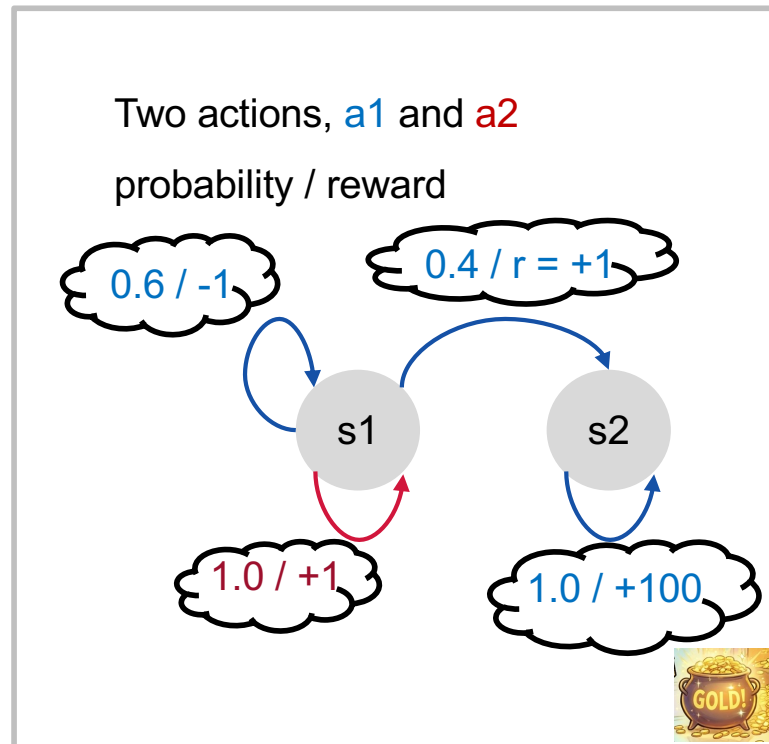
<https://www.youtube.com/watch?v=V1eYniJ0Rnk&t=27s>

Exploration vs. Exploitation



Set $\pi_{\epsilon\text{-greedy}} = \begin{cases} \arg \max_a Q(s, a) & \text{with prob } (1 - \epsilon) \\ \text{a random action} & \text{with prob } \epsilon \end{cases}$

Q-Learning: the need for Exploration



Initialize $Q(s, a) = 0 \forall s, a$

Loop:

Take action $\arg \max_a Q(s, a)$ observe reward r & next state s'

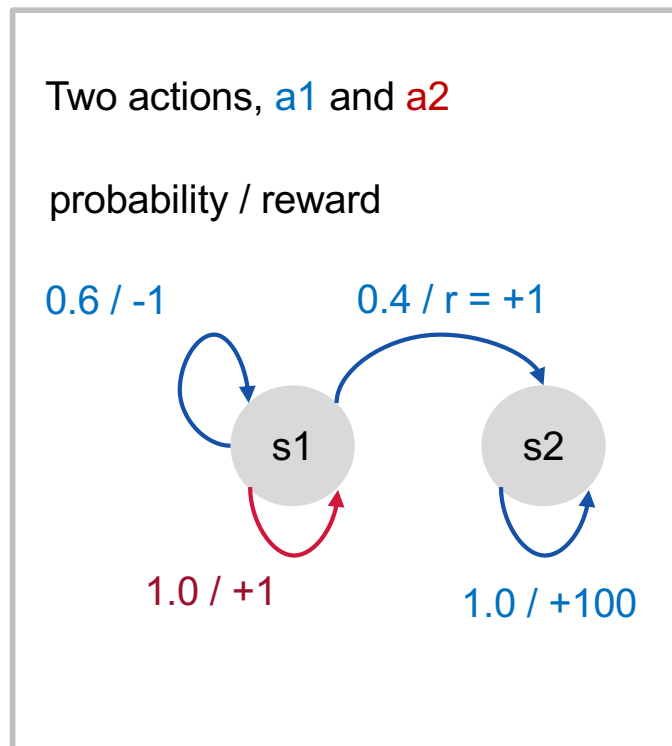
Update $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$

Assume $\alpha = 1$

How would Q learning behave if the first update were $Q(s1, a1)$, $r(s1, a1) = -1$, and actions were selected based on $\arg \max_a Q(s, a)$?



Q-Learning: the need for Exploration



$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + (\alpha) \left[r + \gamma \max_{a'} Q(s', a') \right]$$

$$Q(s1, a1) = 0$$

$$Q(s1, a2) = 0$$

$$Q(s2, a1) = 0 \quad \leftarrow \text{will never change}$$

$$Q(s1, a1) = -1$$

$$Q(s1, a2) > Q(s1, a1), \rightarrow a2 \rightarrow$$

$$Q(s1, a2) = 1$$

$$Q(s1, a2) > Q(s1, a1) \text{ still, } \rightarrow a2$$

$$Q(s1, a2) = 1 + \gamma \dots$$

...

Keep selecting a2

(not optimal, need exploration...)

How would Q learning behave if the first update were $Q(s1, a1)$ and $r(s1, a1) = -1$, actions were selected based on $\arg \max_a Q(s, a)$?



How to Explore?

Several schemes for forcing exploration

- **Simplest: random actions (ϵ -greedy)**
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1-\epsilon$, act on current policy

$$\text{Set } \pi_{\epsilon\text{-greedy}} = \begin{cases} \arg \max_a Q(s, a) & \text{with prob } (1 - \epsilon) \\ \text{a random action} & \text{with prob } \epsilon \end{cases}$$

How to Explore?

Several schemes for forcing exploration

- **Simplest: random actions (ϵ -greedy)**
 - Every time step, flip a coin
 - With (small) probability ϵ , act randomly
 - With (large) probability $1-\epsilon$, act on current policy
- **Problems with random actions?**
 - You do eventually explore the space, but keep thrashing around once learning is done
 - One solution: lower ϵ over time
 - Another solution: *optimal* exploration policies





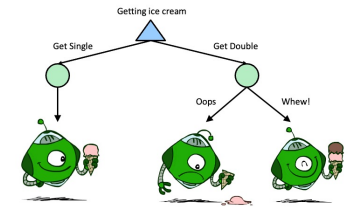
Regret: Formalizing Optimal Exploration

- Even if you learn the optimal policy, you still make mistakes along the way!
- Regret is a measure of your total mistake cost: the difference between your (expected) rewards and optimal (expected) rewards
- Minimizing regret goes beyond learning to be optimal – it requires optimally learning to be optimal
- Example: random exploration and (more sophisticated) exploration functions both end up optimal, but random exploration has higher regret

$$R(T) = \sum_{t=1}^T \left(V^{\pi^*}(s_1) - V^{\pi_t}(s_1) \right)$$

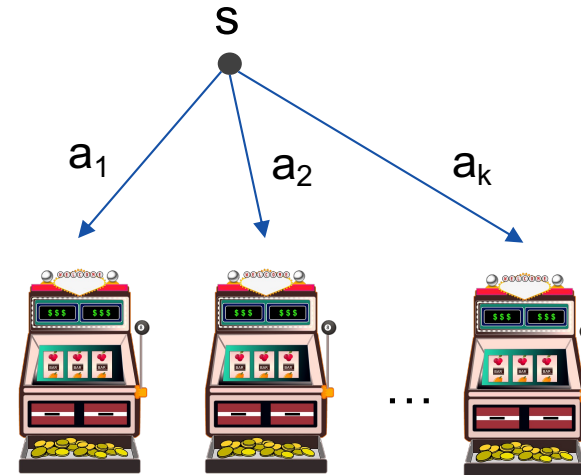


Multi-Armed Bandit Problem Formulation (one step RL)



K slot machines (different win probabilities)

Sampling action a is like pulling a slot machine arm with random payoff r_n



Goal: find an arm-pulling strategy that maximizes the expected total reward at time n

Procedure: at time step n , pick an arm a_n based on what happened so far and receive reward r_n



D2380, Re



UCB Algorithm

Optimizes Regret!

In Q-learning

$$a = \arg \max_a \left[Q(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}} \right]$$

1. Play each machine once
2. Loop: take action a_n & update reward

$$a_n = \arg \max_a Q(a) + \sqrt{\frac{2 \ln n}{n(a)}}$$

$Q(a)$: average reward for trying action a so far

$n(a)$: number of pulls of arm a so far

Exploitation:

favors actions that looked good historically

Exploration:

actions get an exploration bonus that grows with $\ln(n)$

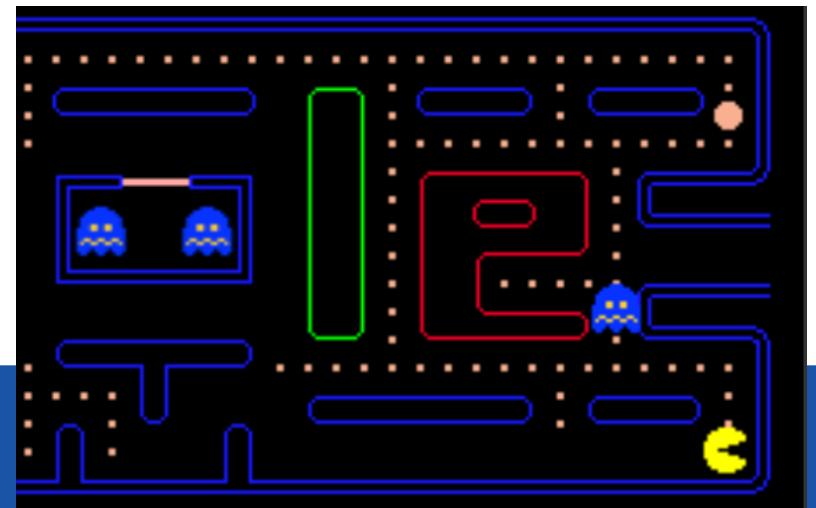
Note: assuming rewards in $[0, 1]$!

Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2), 235-256.



Reward Shaping

- In complex environments, if the agent only receives a reward at the very end of a task (sparse reward), learning is difficult
- Reward shaping is used to accelerate the learning process by providing an agent with more frequent and informative feedback
 - Improve sample efficiency
 - Guide exploration
 - Prevent local optima
- Like a heuristic for search problems





Reward function examples

A standard “recipe” for immediate rewards:

$$r_t = \sum_{k=1}^K w_k f_k(s_t, a_t)$$

where:

- w_k is the scalar that tunes the importance of that feature
- $f_k(s_t, a_t)$ is a feature function evaluating one aspect of task-specific behaviors (e.g. distance to the goal, current speed, number of collisions with other agents, etc.).



nature

Explore content ▾ About the journal ▾ Publish with us ▾ Subscribe

[nature](#) > [articles](#) > article

Article | Published: 09 February 2022

Outracing champion Gran Turismo drivers with deep reinforcement learning

[Peter R. Wurman](#) , [Samuel Barrett](#), [Kenta Kawamoto](#), [James MacGlashan](#), [Kaushik Subramanian](#), [Thomas J. Walsh](#), [Roberto Capobianco](#), [Alisa Devlic](#), [Franziska Eckert](#), [Florian Fuchs](#), [Leilani Gilpin](#), [Piyush Khandelwal](#), [Varun Kompella](#), [HaoChih Lin](#), [Patrick MacAlpine](#), [Declan Oller](#), [Takuma Seno](#), [Craig Sherstan](#), [Michael D. Thomure](#), [Houmehr Aghabozorgi](#), [Leon Barrett](#), [Rory Douglas](#), [Dion Whitehead](#), [Peter Dürr](#), ... [Hiroaki Kitano](#) [+ Show authors](#)

[Nature](#) **602**, 223–228 (2022) | [Cite this article](#)

42k Accesses | 319 Citations | 2299 Altmetric | [Metrics](#)

Abstract

Many potential applications of artificial intelligence involve making real-time decisions in physical systems while interacting with humans. Automobile racing represents an extreme example of these conditions; drivers must execute complex tactical manoeuvres to pass or block opponents while operating their vehicles at their traction limits¹. Racing simulations, such as the PlayStation game Gran Turismo, faithfully reproduce the non-linear control challenges of real race cars while also encapsulating the complex multi-agent interactions. Here we describe how we trained agents for Gran Turismo that can compete with the world's best e-sports drivers. We combine state-of-the-art, model-free, deep reinforcement learning algorithms with mixed-scenario training to learn an integrated control policy that combines exceptional speed with impressive tactics. In addition, we construct a reward function that enables the agent to be competitive while adhering to racing's important, but under-specified, sportsmanship rules. We demonstrate the capabilities of our agent, Gran Turismo Sophy, by winning a head-to-head competition against four of the world's best Gran Turismo drivers. By describing how we trained championship-level racers, we demonstrate the possibilities and challenges of using these techniques to control complex dynamical systems in domains where agents must respect imprecisely defined human norms.

The reward function was a hand-tuned linear combination of these reward components:

- course progress
- off-course penalty
- wall penalty
- tyre-slip penalty
- passing bonus
- any-collision penalty
- rear-end penalty
- unsporting-collision penalty

<https://www.nature.com/articles/s41586-021-04357-7>

DD2380, Reinforcement Learning



Reward shaping challenges

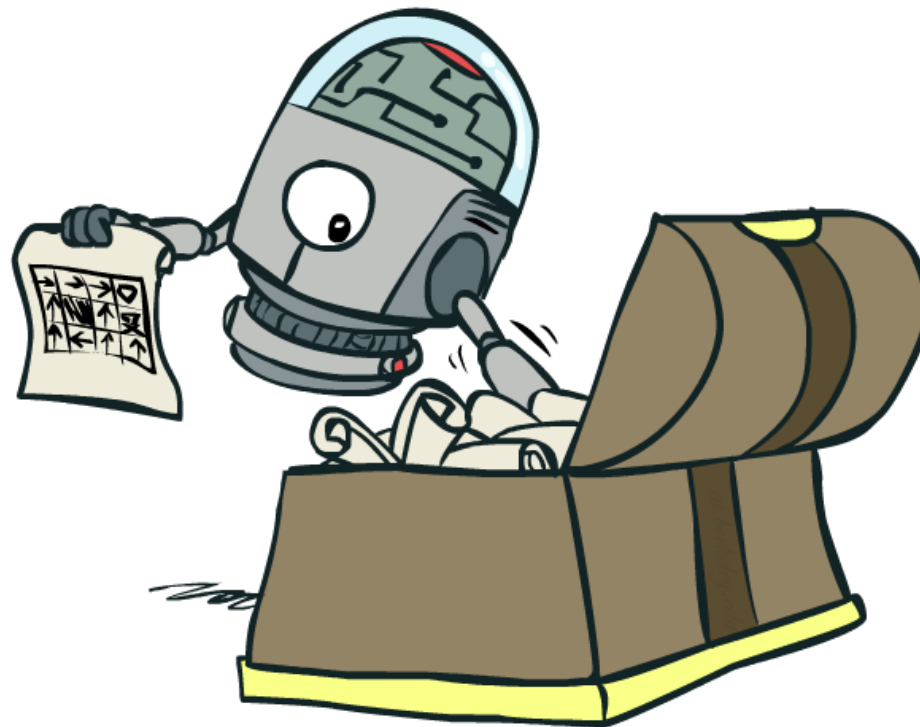
When rewards are misspecified



<https://openai.com/index/faulty-reward-functions/>

Policy Search

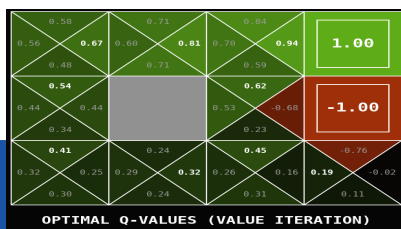
Directly optimizing over policies
Not part of course, but cool...





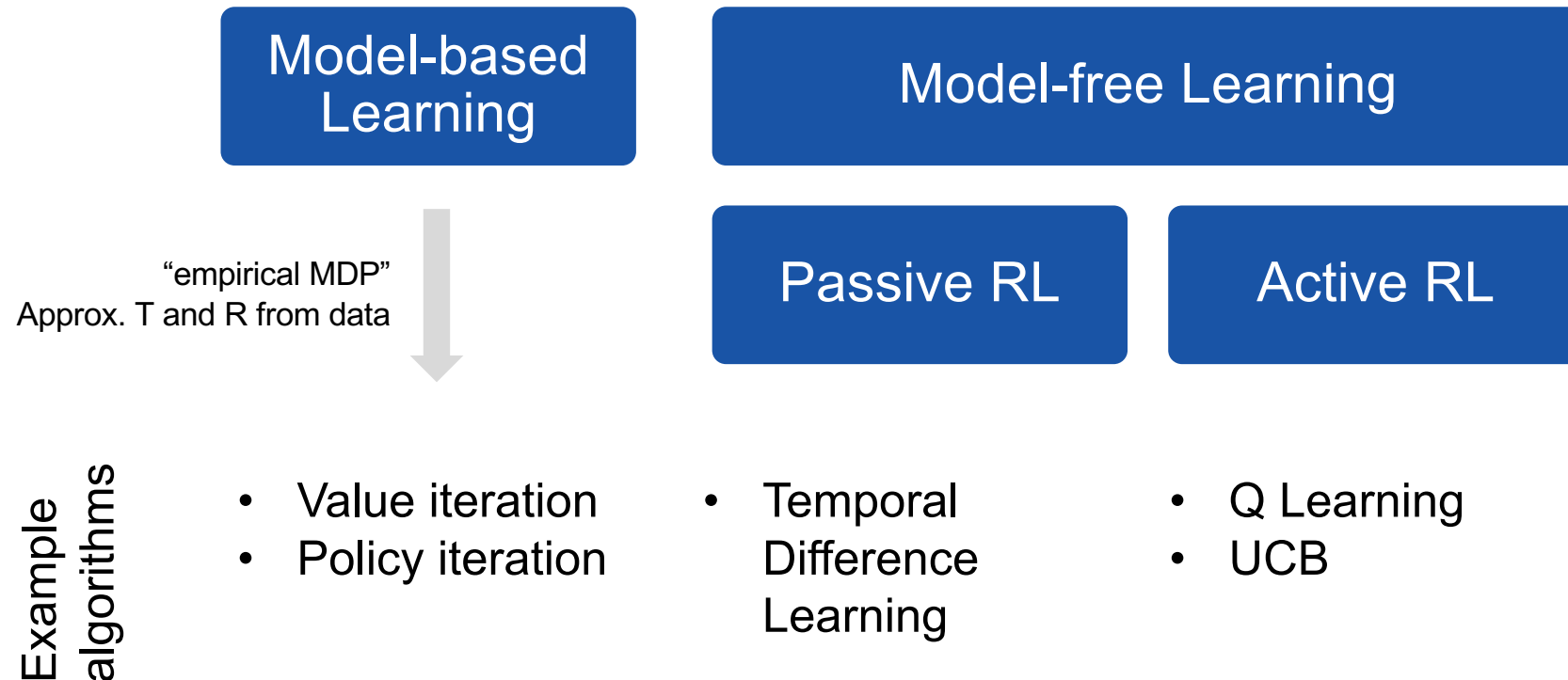
Scaling up: From Tables to Neural Networks

Feature / Dimension	Tabular RL	Neural Network RL (Deep RL)
Representation	Exact lookup tables (or entries).	Function approximation via weights and layers.
Generalization	None. Each state/action is treated independently.	High. Can infer values for unseen states based on similarity.
State Space Scaling	Fails in large or continuous state spaces (memory explosion).	Handles massive, high-dimensional, or continuous spaces.
Stability	Guaranteed convergence under standard conditions.	Prone to instability and divergence
		Experience Replay buffer to decorrelate transitions.
		Separate Target Networks to stabilize updates.





How to find optimal policies without knowing T and R ?





Credits

Based partly on materials from:

- Iolanda Leite, KTH
- Rika Antonova, KTH
- Pieter Abbeel & Dan Klein, UC Berkeley
- Alan Fern, Oregon State
- Shipra Agrawal, Columbia University
- Fei Fang and Dave Touretzky, CMU



Thank You!



Bellman equation

Bellman equation

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U_i(s')$$

Converges to unique optimal solution

Stop iterations when largest change in utility for any state is small enough

Can show that

$$\|U_{i+1} - U_i\| < \epsilon \frac{1-\gamma}{\gamma} \Rightarrow \|U_{i+1} - U\| < \epsilon$$



Value iteration

Key insight: Utility of a state is immediate reward plus discounted expected utility of next states

$$U(s) = R(s) + \gamma \max_a \sum_{s'} T(s, a, s') U(s')$$

(assuming that we choose the optimal policy)

Idea: Iterate

- Calculate utility of each state
- Use utilities to select optimal decision in each state



Algorithm: Value iteration

Initialize $U(s)$ arbitrary for all s

Loop until policy has converged

 loop over all states, s

 loop over all actions, a

$$Q(s, a) := R(s) + \gamma \sum_{s'} T(s, a, s') * U(s')$$

 end

$$U(s) := \max_a Q(s, a)$$

 end

end



Policy iteration

Alternative to value iteration

Choose an arbitrary policy

Loop until policy does not change any more

- Compute the value function, $V(s)$ given the policy, known as “policy evaluation”

$$V(s) = \sum_{s' \in \mathcal{S}} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V(s')], \forall s \in \mathcal{S}$$

- Given this value function, improve the policy for each state

$$\pi(s) \leftarrow \arg \max_a \sum_{s' \in \mathcal{S}} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$



LLMs and RL

- Classic LLM training
 - Supervised learning: “part of text” → “predict next word”
- RL
 - State: “Prompt and output so far”
 - Action: “next word”
 - Reward: “from separate NN trained on human preferences”
 - PPO updates policy (LLM weights) to produce better scores
 - Next level: “reasoning”, “did I arrive at answer”?